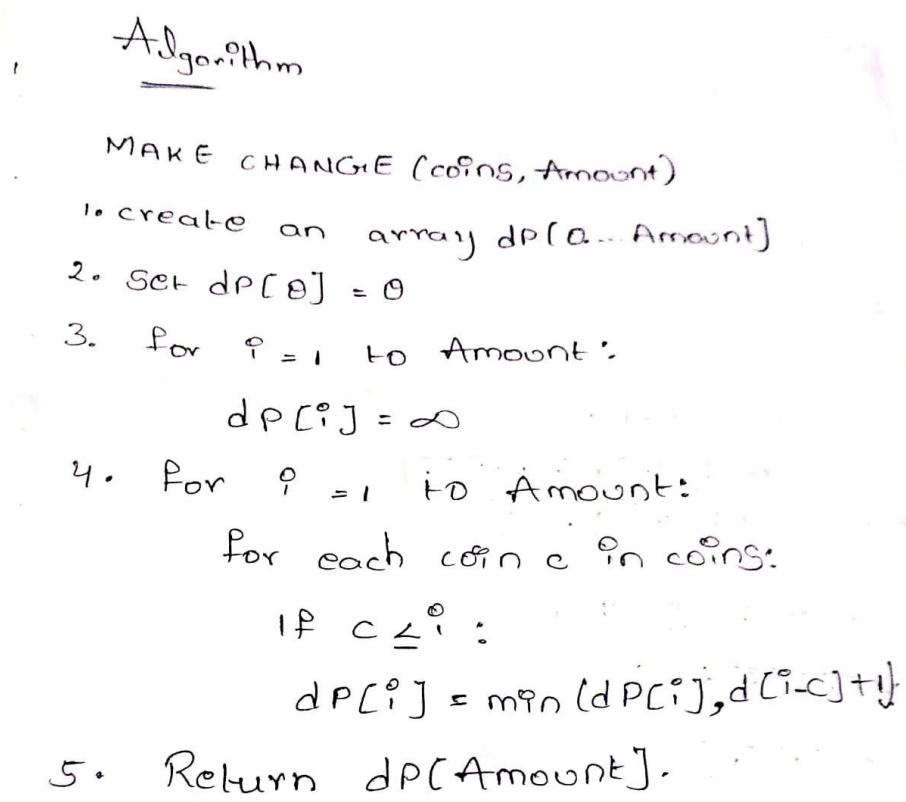
Practical 7

AIM:

To Implementation of making a change problem using dynamic programming technique by determining the minimum number of coins required to make a given amount from a set of available coin denominations.

ALGORITHM:



CODE:

```
#include <iostream>
#include <vector>
```

```
#include <climits>
using namespace std;

int main() {
    int n, amount;

    cout << "Enter number of coins: ";
    cin >> n;

    vector<int> coins(n);

    cout << "Enter coin values: ";
    for (int i = 0; i < n; i++)
        cin >> coins[i];

    cout << "Enter amount: ";
    cin >> amount;

    vector<int> dp(amount + 1, INT_MAX);

    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i && dp[i - coins[j]] != INT_MAX) {
                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
            }
        }
    }

    if (dp[amount] == INT_MAX)
        cout << "Change cannot be made";
    else
        cout << "Minimum number of coins = " << dp[amount];

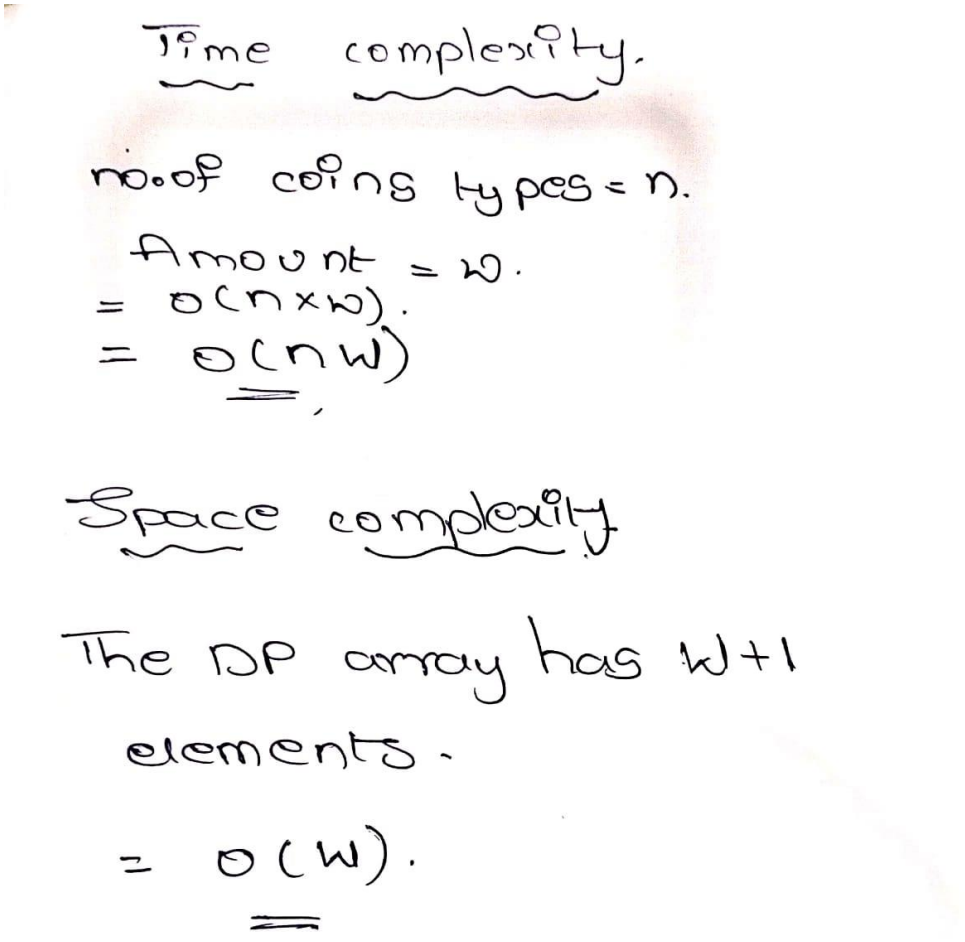
    return 0;
}
```

OUTPUT:

```
Output

Enter number of coins: 3
Enter coin values: 2 4 6
Enter amount: 4
Minimum number of coins = 1
```

TIME COMPLEXITY:



CONCLUSION:

The Making Change problem can be efficiently solved using Dynamic Programming by storing the minimum number of coins required for every smaller amount and using these results to calculate the answer for the target amount.